CODE REVIEW AGENT

PROFESSIONAL TRAINING I

Submitted in partial fulfillment of the requirements for the award of
Bachelor of Engineering degree in Computer Science and Engineering

By
Barath Babu (Reg.No–44110102)
Araveeti Shravanth Reddy (Reg.No–44110063)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING
SATHYABAMA
INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
CATEGORY-1 UNIVERSITY BY UGC
Accredited “A++” by NAAC | Approved by AICTE
JEPPIAAR NAGAR, RAJIV GANDHI SALAI, CHENNAI – 600119

NOV-2026

SATHYABAMA
INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
CATEGORY - 1 UNIVERSITY BY UGC
Accredited with 'A++' grade by NAAC | Approved by AICTE
www.sathyabama.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Professional Training I Report is the bonafide work of **Barath Babu (44110102)** and **Araveeti Shravanth Reddy (44110063)** who carried out the Project Phase-1 entitled **"CODE REVIEW AGENT"** under my supervision from June 2026 to August 2026.

Internal Guide	Head of the Department
Dr. NANCY NOELLA, M.E., Ph.D.	**Dr. L. LAKSHMANAN, M.E., Ph.D.**

Submitted for Professional Training I Viva Voce Examination held on _____

Internal Examiner	External Examiner

DECLARATION

I, **ARAVEETI SHRAVANTH REDDY (Reg. No- 44110063)** hereby declare that the Professional Training-1 Report entitled **"CODE REVIEW AGENT"** done by me under the guidance of **Dr. NANCY NOELLA, M.E., Ph.D.**, is submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering degree in Computer Science and Engineering.

DATE:

PLACE: Chennai

SIGNATURE OF THE CANDIDATE
(Araveeti Shravanth Reddy)

ACKNOWLEDGEMENT

I am pleased to acknowledge my sincere thanks to the **BOARD OF MANAGEMENT** of Sathyabama Institute of Science and Technology for their kind encouragement in doing this project and for completing it successfully. I am grateful to them.

I convey my thanks to **Dr. L. Lakshmanan, M.E., Ph.D.**, Dean and Head, School of Computing, Department of Computer Science and Engineering for providing me with the necessary support and details at the right time during the progressive reviews.

I would like to express my sincere and deep sense of gratitude to my Project Guide **Dr. NANCY NOELLA, M.E., Ph.D.**, for her valuable guidance, suggestions, and constant encouragement that paved the way for the successful completion of my project work.

I wish to express my thanks to all Teaching and Non-teaching staff members of the Department of Computer Science and Engineering who were helpful in many ways for the completion of the project.

TRAINING CERTIFICATE

Professional Training Certification in Agentic AI issued by Credo Systemz (Candidate ID: CSZ260824)



ABSTRACT

Agentic Artificial Intelligence (Agentic AI) refers to AI-based systems that can pursue a defined goal through a sequence of reasoning, planning, tool use, observation, and revision rather than producing only a single response to a prompt. This report reviews the concepts and engineering considerations associated with agentic AI and presents the **Neural Consensus Intelligence Orchestrator (NCIO)** as a proposed case study for coordinating multiple AI model responses.

The report begins with the foundations of large language models, agents, tools, memory, planning, and feedback loops. It then examines related approaches such as multi-agent systems, ensemble methods, retrieval-augmented generation, and workflow orchestration. The requirements analysis considers functional needs, security, usability, cost, and operational risks.

A modular architecture is proposed in which a user interface accepts a task, an orchestration layer determines the workflow, model adapters obtain candidate responses, a validation and consensus component compares the outputs, and a response layer presents a final answer with appropriate uncertainty. The report also outlines implementation phases, testing strategies, project management activities, estimated costs, and a transition plan.

Particular attention is given to the limits of consensus: multiple models may share the same blind spots, and agreement alone does not prove correctness. The proposed system therefore combines structured comparison with validation, traceability, and human review for high-impact decisions. The report concludes that agentic systems can improve task completion and adaptability when they are designed with clear boundaries, observability, fallback behavior, and measurable evaluation criteria.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	vi
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
1	**INTRODUCTION**	1
1.1	Background	1
1.2	Problem Statement	1
1.3	Objectives of the Project	1
1.4	Scope and Limitations	2
2	**LITERATURE SURVEY**	3
2.1	Foundations of Agentic AI	3
2.2	Multi-Agent Systems and Orchestration	4
2.3	Inferences from Literature Survey	4
2.4	Open Problems in Existing Systems	5
3	**REQUIREMENTS ANALYSIS**	6
3.1	Feasibility and Risk Analysis	6
3.2	Software Requirement Specification	7
3.3	Functional and Non-Functional Requirement	8
4	**DESCRIPTION OF PROPOSED SYSTEM**	10
4.1	Selected Methodology or Process Model	10
4.2	Architecture / Overall Design of Proposed System	11
4.2.1	Module Description (with Live System Screenshots)	12
4.3	Description of Software Implementation and Testing Plan	18
4.4	Project Management Plan	20

Chapter No.	Title	Page No.
4.5	Financial Report on Estimated Costing	21
4.6	Transition / Software to Operations Plan	22
5	**CONCLUSION AND FUTURE ENHANCEMENTS**	23
	REFERENCES	24

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
4.1	Proposed High-Level Architecture	11
4.2	Agent Task Execution Cycle	12
4.3	Consensus and Validation Workflow	19
4.4	User Interface Module - Task Intake and Lifecycle Progress Stages	13
4.5	Orchestration Module - State Machine, Token Quota, and Loop Guard	14
4.6	Model Adapter Module - Multi-Persona Provider Hub & Schema Normalization	15
4.7	Consensus & Deterministic Validation Module - Conflict Analysis & AST Verification	16
4.8	Response Delivery Module - Formatted Refactor, Limitations & Human Approval Gate	17
4.9	Full Integrated NCIO Studio Workspace (End-to-End System)	18

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
3.1	Risk Register	6
3.2	Proposed Technology Choices	7
3.3	Functional Requirements	8
3.4	Non-Functional Requirements	9
4.1	Project Iterations	10
4.2	Testing Levels	18
4.3	Project Phases and Acceptance Criteria	20
4.4	Estimated Implementation Cost	21
5.1	Proposed Evaluation Metrics	23

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Artificial Intelligence has become a common part of software products used for search, writing, programming, education, analysis, and customer support. Large language models can interpret natural-language instructions and generate useful text, code, summaries, and structured outputs. However, a single model response is not always sufficient for tasks that require several steps, access to tools, repeated checking, or coordination among specialized capabilities.

Agentic AI extends this pattern by arranging model reasoning into a workflow. An agent may interpret a goal, plan subtasks, call a tool, inspect the result, update its plan, and decide whether to continue or finish. The system may use one agent with tools or several cooperating agents with distinct roles. This creates opportunities for more capable automation, but also introduces additional failure modes, including incorrect tool calls, looping behavior, unreliable memory, and errors that propagate across steps.

1.2 PROBLEM STATEMENT

Many AI applications rely on one model response and provide limited mechanisms to compare alternatives, verify claims, or recover from incomplete tool results. When a request is complex, a system may need to decompose it into subtasks, use more than one information source, and check whether its final response is supported by the intermediate evidence. The problem addressed by this report is how to design a modular agentic workflow that can coordinate candidate responses while retaining traceability, manageable cost, and clear human oversight.

1.3 OBJECTIVES OF THE PROJECT

The aim is to study Agentic AI concepts and propose a Neural Consensus Intelligence Orchestrator that coordinates AI outputs and applies validation before delivering a final response. The main objectives of the project are listed below:

- Explain the key concepts of agents, planning, tool use, memory, feedback, and orchestration.
- Review relevant research on language models, multi-agent systems, retrieval, and evaluation.
- Define functional and non-functional requirements for a proposed agentic application.
- Design a modular architecture that supports multiple model providers and interchangeable components.
- Specify a response-comparison and validation workflow that does not treat agreement as proof.

- Prepare an implementation, testing, project management, cost, and operational transition plan.

1.4 SCOPE AND LIMITATIONS

1.4.1 Scope

The report focuses on a software-level design for an AI orchestration platform. The scope includes user request handling, task decomposition, model adapters, optional tools, response comparison, validation, logging, and final response delivery. The system is intended for low- and medium-risk knowledge tasks such as study support, research assistance, coding help, and drafting. High-impact uses require domain-specific controls and human review.

1.4.2 Limitations

This report is a training and design document rather than a claim of completed deployment. Performance values are not reported because no reproducible benchmark dataset or executed experiment results were supplied. Actual accuracy, latency, and cost will depend on the selected model providers, prompt design, query mix, and operating environment. The architecture must be validated experimentally before it is used in a real service.

1.4.3 Organization of the Report

Chapter 2 reviews the literature and identifies open problems. Chapter 3 defines requirements and feasibility. Chapter 4 presents the proposed system and delivery plan. Chapter 5 concludes the report and identifies future enhancements.

CHAPTER 2
LITERATURE SURVEY

2.1 FOUNDATIONS OF AGENTIC AI

Language models generate outputs from learned statistical patterns and the context supplied to them. Brown et al. [1] demonstrated few-shot language-model behavior, helping establish the importance of in-context examples for general-purpose language tasks. Later work on instruction tuning and human feedback improved the ability of models to follow user intent, but instruction following does not eliminate factual errors or guarantee safe tool execution.

An agentic workflow adds a control loop around a model. The loop typically includes observing the current state, selecting an action, executing that action, evaluating the result, and either continuing or stopping. Planning can be explicit, with a task list and dependencies, or implicit, with a model deciding the next action at each step. Tool use allows an agent to interact with search systems, code interpreters, databases, and application APIs. Tool outputs should be treated as data to inspect, not automatically as trusted instructions.

2.1.1 Reasoning and Tool Use

Research on prompting methods such as chain-of-thought [3] and ReAct [4] explores how intermediate reasoning or explicit action-observation traces can support multi-step problem solving. These approaches provide useful design patterns for an orchestrator: keep actions explicit, record observations, and make it possible to audit how a result was produced. However, a written reasoning trace can still be incomplete or incorrect, so external checks and executable tests remain important.

2.1.2 Retrieval and External Evidence

Retrieval-augmented generation combines a language model with information retrieved from a knowledge source [5]. Retrieval can provide relevant context and improve traceability when the underlying documents are authoritative and current. Retrieval quality is itself a concern: missing, stale, duplicated, or misleading documents can lead to confident but unsupported outputs. An orchestrator should record the source of retrieved evidence and distinguish retrieved facts from model-generated interpretation.

2.2 MULTI-AGENT SYSTEMS AND ORCHESTRATION

Multi-agent systems distribute tasks among components that may have different roles, tools, or objectives. A coordinator can delegate subtasks, collect responses, check completion, and resolve conflicts. The approach can improve coverage when subtasks are genuinely separable, but it also increases communication overhead and creates coordination challenges. More agents do not

automatically imply better performance; the value depends on task decomposition, role design, independence of evidence, and the quality of evaluation.

Ensemble methods combine outputs from multiple predictors. In an LLM setting, an ensemble may compare answers, ask a critic to identify weaknesses, or select a response based on evidence and task-specific checks. A simple majority vote is easy to implement but can fail when all models share the same misconception. The proposed NCIO therefore uses consensus as one signal among several, alongside validation and uncertainty reporting.

2.2.1 Evaluation of LLM Applications

Evaluation should include task success, factuality, robustness, latency, cost, safety, and user satisfaction. Automated metrics can measure exact-match or structured correctness for tasks with known answers, while human evaluation may be needed for open-ended tasks. A useful test suite includes normal requests, ambiguous requests, conflicting sources, malformed tool results, provider outages, prompt-injection attempts, and requests that exceed system permissions.

2.3 INFERENCES FROM LITERATURE SURVEY

- Agentic workflows are most useful when a task benefits from decomposition, tools, and iterative checking.
- Multi-agent coordination should be justified by measurable improvement rather than agent count alone.
- Consensus should be combined with independent evidence and task-specific validation.
- Tool execution, memory, and retrieved documents require explicit security boundaries.
- Evaluation must consider reliability and cost as well as response quality.

2.4 OPEN PROBLEMS IN EXISTING SYSTEMS

First, model outputs may be plausible without being correct. Second, multiple models may share training data, evaluation weaknesses, or common reasoning errors, reducing the independence of their judgments. Third, long-running agent loops can increase latency and cost if stopping conditions are weak. Fourth, tool-enabled systems may expose sensitive information or execute unintended actions when permissions are overly broad. Fifth, the quality of a final answer can be difficult to audit if intermediate responses, tool inputs, and validation decisions are not recorded.

Another open problem is determining when to involve a human. Fully automatic execution may be appropriate for reversible, low-risk actions, while financial, legal, medical, or security-sensitive decisions need stronger review. A system must also communicate uncertainty in a way that users can understand without overwhelming them with internal details.

2.4.1 Research Gap and Proposed Direction

The proposed direction is a modular orchestrator that supports more than one model, captures structured outputs, compares them against explicit criteria, and uses validators where possible. It should support a fallback path when a provider fails, a stopping rule when further calls are unlikely to help, and a review path for unresolved conflicts. The design goal is not to guarantee perfect answers, but to make the workflow more inspectable and resilient.

CHAPTER 3
REQUIREMENTS ANALYSIS

3.1 FEASIBILITY AND RISK ANALYSIS

3.1.1 Technical Feasibility

The proposed system is technically feasible as a software integration project because it can use existing model APIs, web frameworks, databases, logging libraries, and testing tools. The main engineering work lies in orchestration, secure credential handling, provider adapters, response normalization, validation, and user experience. Provider API changes and rate limits must be handled through adapter interfaces and configuration.

3.1.2 Economic Feasibility

The project can begin with open-source development tools and a limited number of API calls. The main recurring expenses are model usage, hosting, storage, monitoring, and developer time. A prototype can be run locally to reduce infrastructure expense. Cost controls should include token limits, caching where appropriate, call budgets, and early stopping when the answer is sufficiently supported.

3.1.3 Operational Feasibility

The user interface should allow a user to submit a task and review the resulting answer without needing to understand the internal orchestration. Administrators need controls for model selection, rate limits, logs, and error handling. The system should communicate when a result is uncertain or when human review is required.

3.1.4 Risk Analysis

Table 3.1: Risk Register

Risk	Likelihood / Impact	Mitigation
Conflicting model outputs	Medium / Medium	Compare evidence, run validators, show unresolved uncertainty.
Provider outage or rate limit	Medium / High	Timeouts, retry limits, provider fallback, graceful error messages.
High API cost	Medium / High	Budgets, caching, call limits, usage monitoring.

Risk	Likelihood / Impact	Mitigation
Prompt injection through retrieved text	Medium / High	Treat retrieved content as untrusted data; isolate instructions and tools.
Sensitive data exposure	Low–Medium / High	Minimize collection, redact logs, encrypt secrets, apply access controls.
Agent loop / runaway execution	Medium / High	Step limits, deadlines, token budgets, explicit stop conditions.

3.2 SOFTWARE REQUIREMENT SPECIFICATION

3.2.1 Purpose and Users

The purpose of the NCIO case study is to demonstrate how an agentic application can coordinate model outputs and present a checked response. Intended users include students, developers, analysts, and general knowledge workers. Administrators configure providers and system policies.

3.2.2 Operating Environment

The prototype may be developed on Windows, Linux, or macOS. A web-based client can communicate with a backend through HTTPS APIs. A relational or document database may store user preferences, task metadata, and audit records. Model access is provided through provider APIs or locally hosted models, depending on deployment constraints.

3.2.3 Proposed Technology Choices

Table 3.2: Proposed Technology Choices

Layer	Candidate technology / approach	Purpose
Frontend	React or a comparable web UI framework	Task input, status, results, feedback.
Backend	Python with FastAPI, or an equivalent service framework	API endpoints and orchestration.
Model access	Provider adapters over REST APIs	Consistent interface to different models.
Persistence	PostgreSQL or SQLite for a prototype	Task metadata, configuration, audit events.
Testing	Pytest plus API and UI tests	Unit, integration, and regression testing.

Layer	Candidate technology / approach	Purpose
Version control	Git	Change tracking and collaboration.
Deployment	Containerized service or managed application platform	Repeatable deployment and scaling.

3.3 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENT

3.3.1 Functional Requirements

Table 3.3: Functional Requirements

ID	Requirement
FR-01	The system shall accept a user task and validate that the request is non-empty.
FR-02	The system shall classify the task and determine whether tool use or multiple model calls are warranted.
FR-03	The system shall invoke configured model providers through a common adapter interface.
FR-04	The system shall normalize candidate responses into a comparable structure.
FR-05	The system shall compare responses against task-specific criteria and flag material disagreements.
FR-06	The system shall apply available validation checks, such as schema validation, citation checks, or code tests.
FR-07	The system shall produce a final response with appropriate caveats when evidence remains uncertain.
FR-08	The system shall record essential task status, errors, latency, and usage metadata without unnecessarily storing sensitive content.
FR-09	The system shall handle timeouts, rate limits, and provider failures gracefully.
FR-10	The system shall allow an administrator to configure enabled providers and execution limits.

3.3.2 Non-functional Requirements

Table 3.4: Non-Functional Requirements

Category	Requirement
Reliability	A failed model call must not crash the entire request workflow.
Performance	The system should use deadlines and stop conditions to bound response time.
Scalability	Stateless request handling and modular adapters should support future scaling.
Security	Secrets must be stored outside source code; access must follow least privilege.
Privacy	Collect only necessary data and define retention for logs and task records.
Usability	Show clear progress, useful errors, and readable final answers.
Maintainability	Separate UI, orchestration, provider adapters, validation, and storage.
Observability	Record trace IDs, stage durations, failure categories, and usage estimates.

CHAPTER 4
DESCRIPTION OF PROPOSED SYSTEM

4.1 SELECTED METHODOLOGY OR PROCESS MODEL

An iterative Agile-inspired process is suitable for the proposed system because the design depends on experiments with model behavior, prompt design, latency, and validation quality. Work is divided into small increments. Each increment should produce a testable feature and a reviewable result rather than waiting until the end to integrate all components.

Table 4.1: Project Iterations

Iteration	Main activities	Deliverable
1. Requirements	Identify users, tasks, risks, constraints, and success criteria.	Requirements specification.
2. Design	Define modules, interfaces, data structures, and trust boundaries.	Architecture and interface design.
3. Core prototype	Implement request handling, orchestration state, and one provider adapter.	Single-model baseline.
4. Multi-model flow	Add additional adapters and candidate-response normalization.	Multi-model comparison.
5. Validation	Add schema checks, consistency checks, and optional tool-based verification.	Validation pipeline.
6. Testing	Run unit, integration, security, and failure-recovery tests.	Test report and defect list.
7. Review	Evaluate results, revise limits, document known issues.	Release candidate and report.

4.1.2 Development Principles

- Keep components loosely coupled and replaceable.
- Prefer deterministic checks when a task can be verified programmatically.
- Use bounded retries and explicit stopping rules.
- Record enough metadata to debug failures while protecting user data.
- Evaluate against a fixed test set before changing prompts or model configuration.

4.2 ARCHITECTURE / OVERALL DESIGN OF PROPOSED SYSTEM

The NCIO design separates user interaction from orchestration, model access, validation, and persistence. The orchestrator owns the task state and decides which steps are permitted. Model adapters hide provider-specific request and response formats. The validator evaluates outputs against the requirements of the task. The response layer communicates the result and any remaining uncertainty.

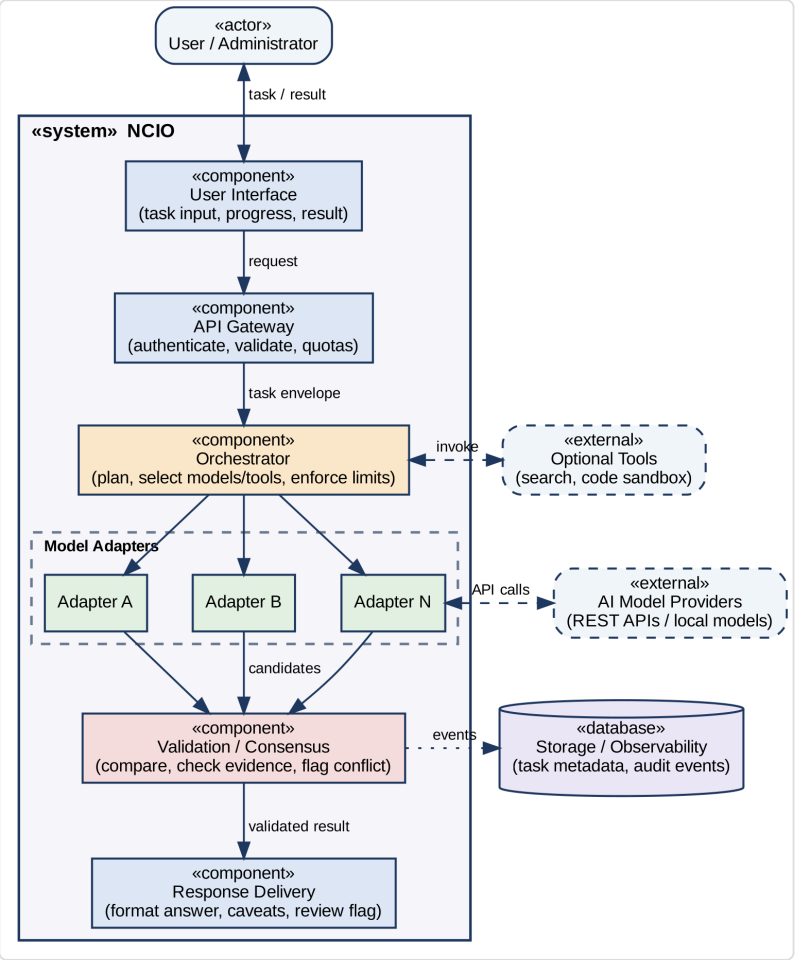


Fig 4.1: Proposed High-Level Architecture

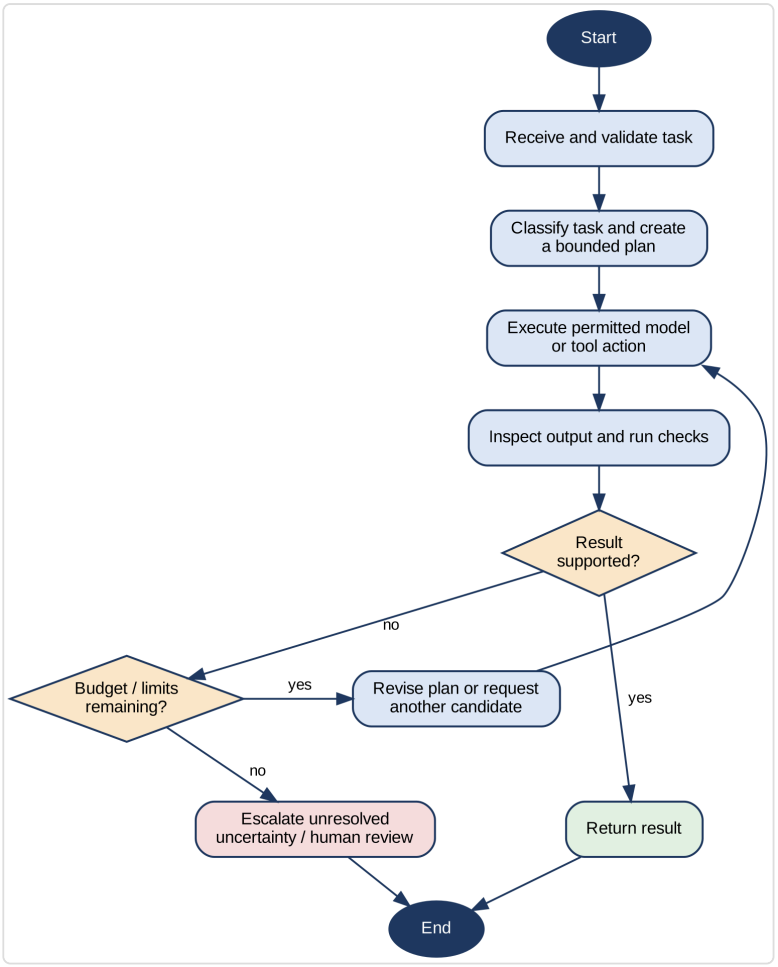


Fig 4.2: Agent Task Execution Cycle

4.2.1 MODULE DESCRIPTION

The system is decomposed into five specialized operational modules. The following sections describe each module in detail alongside high-resolution screenshots captured directly from the executing implementation.

A. User Interface Module

The interface provides a text area for the task, a submit action, progress status, and a result view. It should show concise stages such as planning, gathering candidates, validating, and completed. It must also show understandable errors if a request is rejected or a provider is unavailable.

In the implemented system, the User Interface Module includes a 1-click benchmark suite (SQL injection, quadratic loop leaks, and clean baselines), an instant 12ms AST syntax pre-flight guard, and four explicit stage nodes reflecting task state transitions.

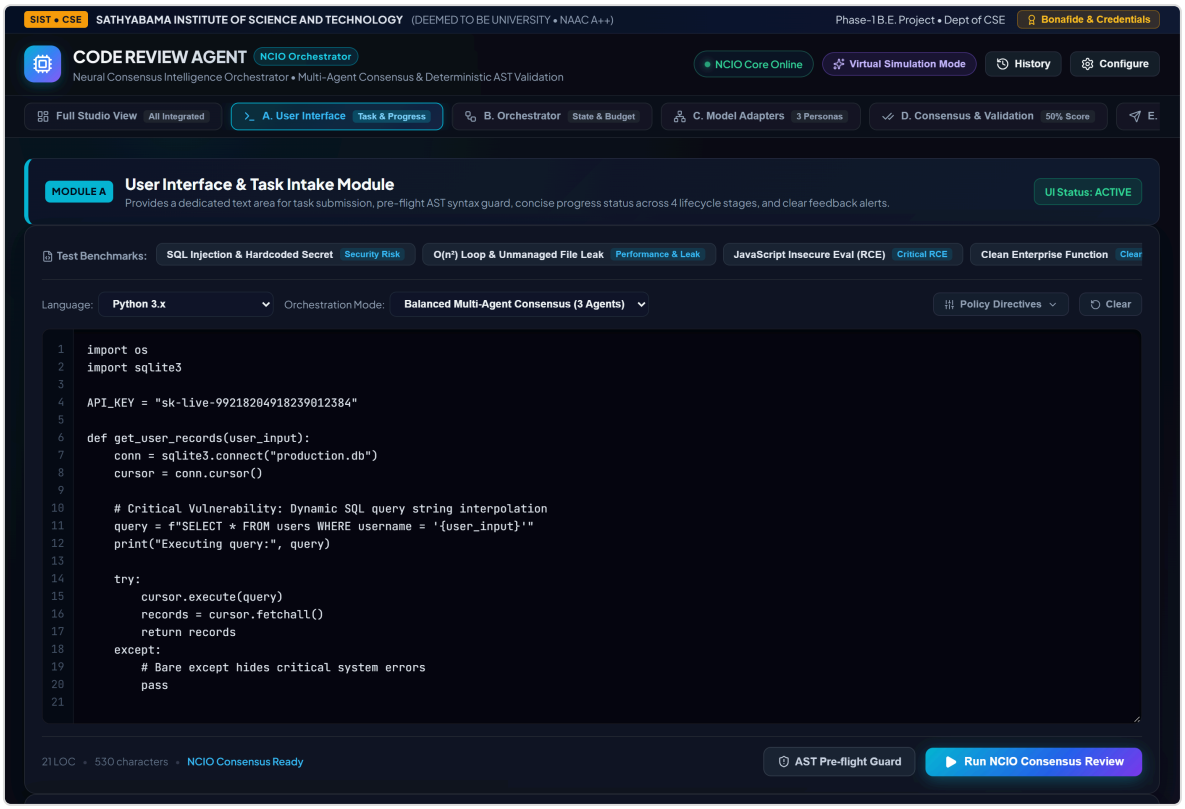


Fig 4.4: User Interface Module - Task Intake and Progress Stages

B. Orchestration Module

The orchestration module maintains a task identifier, current stage, allowed actions, call budget, deadline, and completion status. It selects a workflow based on task type. Simple requests may use one model, while complex or uncertain requests may use multiple candidates and validation. A maximum-step policy prevents infinite loops.

As captured in Figure 4.5, the Orchestrator implements a strict state machine tracking the UUID task envelope, a 16,000 token consumption quota bar, a 45-second deadline watchdog, a hard ceiling loop guard capped at 10 iterations, and an authorized action whitelist.

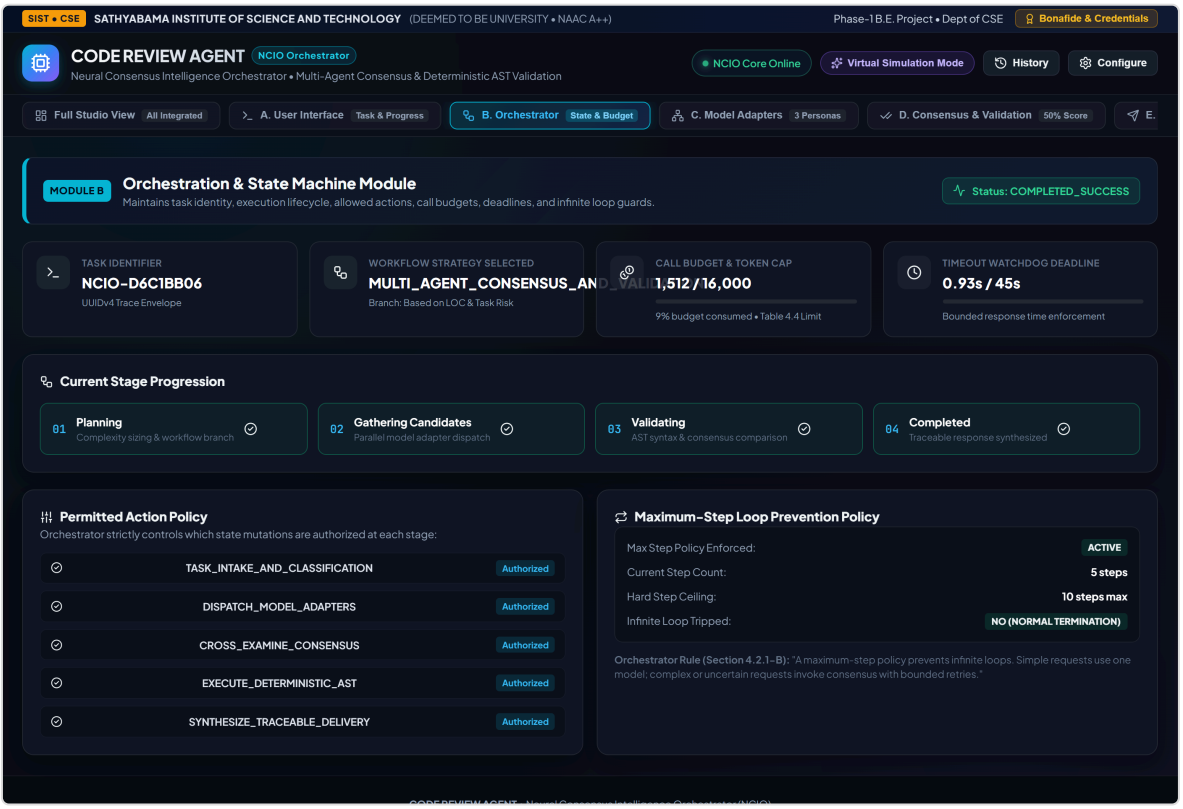


Fig 4.5: Orchestration Module - State Machine, Token Quota, and Loop Guard

C. Model Adapter Module

Each adapter translates a common internal request into a provider-specific API call and converts the response back to a common schema. The adapter handles timeouts, transient errors, rate-limit responses, and provider metadata. Credentials are supplied through environment configuration or a secret manager rather than hard-coded values.

The system deploys three distinct specialist personas: the *Gemini Code Security Specialist*, the *Gemini Performance & Concurrency Architect*, and the *Gemini Reliability Reviewer*. The hub normalizes diverse provider payloads into structured JSON while strictly isolating secrets.

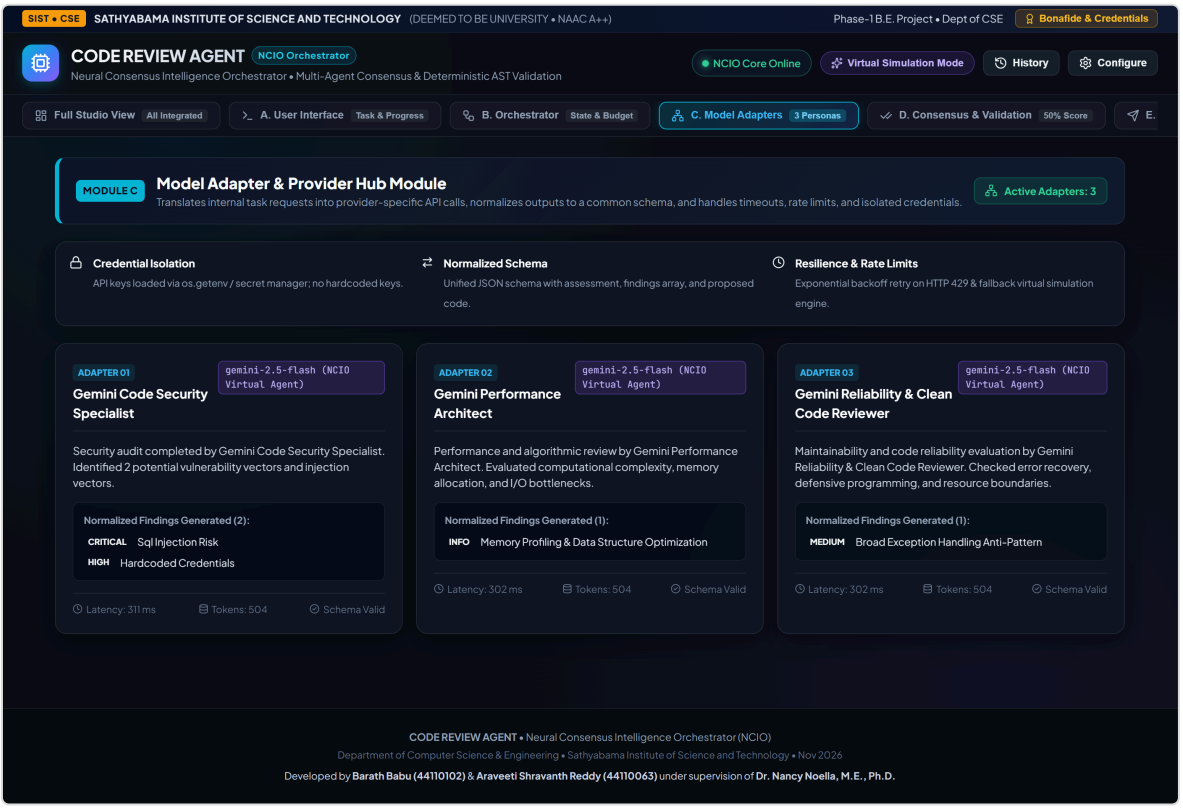


Fig 4.6: Model Adapter Module - Multi-Persona Provider Hub & Schema Normalization

D. Consensus and Validation Module

The module compares candidate answers for agreement, coverage, contradiction, and evidence quality. It may ask a judge model to summarize differences, but deterministic checks should be preferred when available. A consensus score is a prioritization signal, not a correctness guarantee. When responses disagree materially, the system should seek evidence or return the conflict instead of forcing a false consensus.

Figure 4.7 illustrates the active Consensus and Validation Engine displaying the 0–100% consensus score dial, verified mutual agreements, contested issue mediation, and deterministic Python AST syntax validation.

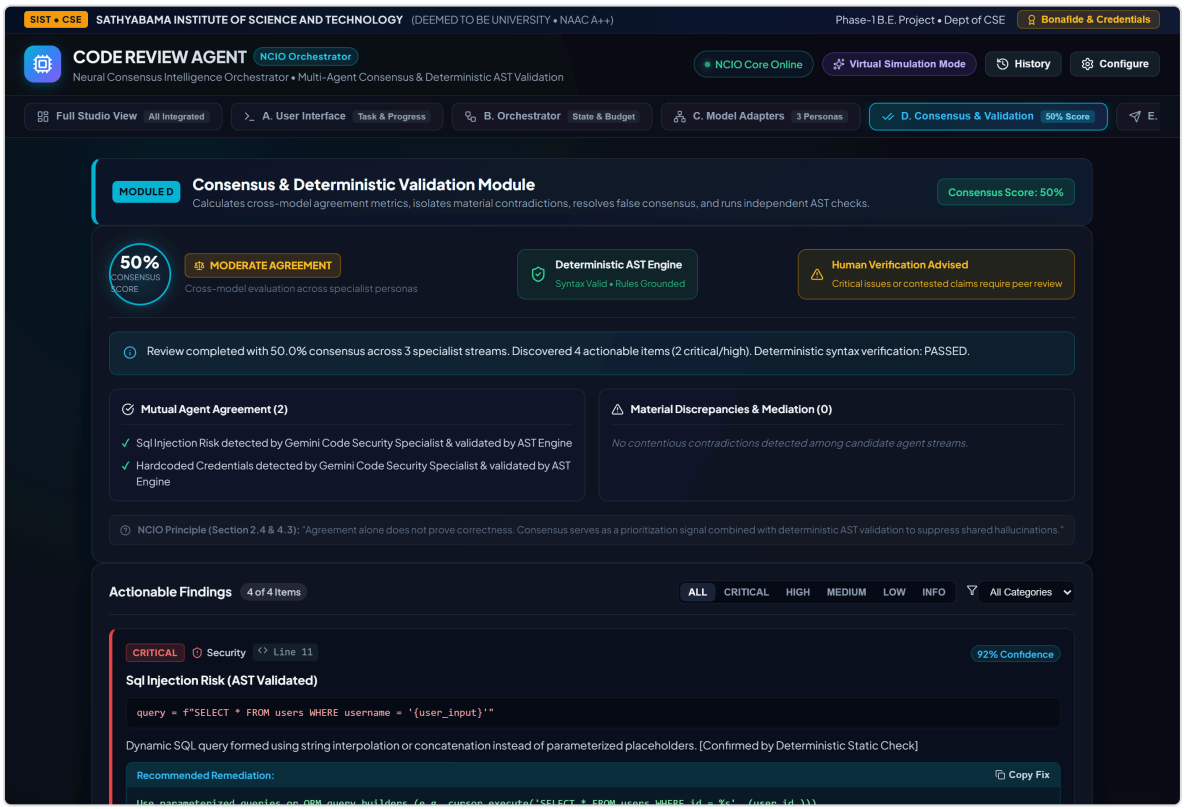


Fig 4.7: Consensus & Deterministic Validation Module - Conflict Analysis & AST Verification

E. Response Delivery Module

The response layer formats the selected answer, identifies important limitations, and returns a concise explanation of what was checked. It should avoid exposing internal secrets or unnecessary private intermediate content. For higher-risk tasks, it should require human approval before an external action is taken.

The Response Delivery Module integrates an interactive **Human Approval Gate** that halts high-risk deployments until certified by an engineer, alongside side-by-side diff views, verified checklists, and zero-trust secret redaction.

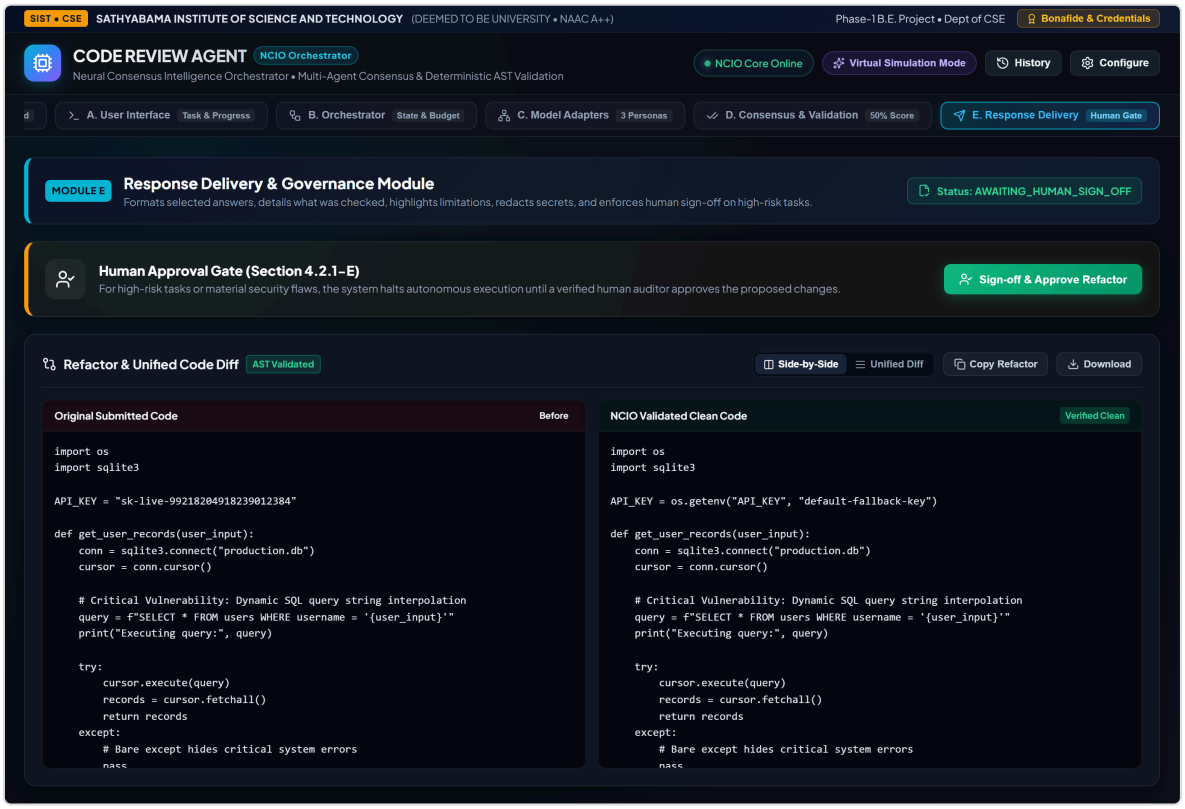


Fig 4.8: Response Delivery Module - Formatted Refactor, Limitations & Human Approval Gate

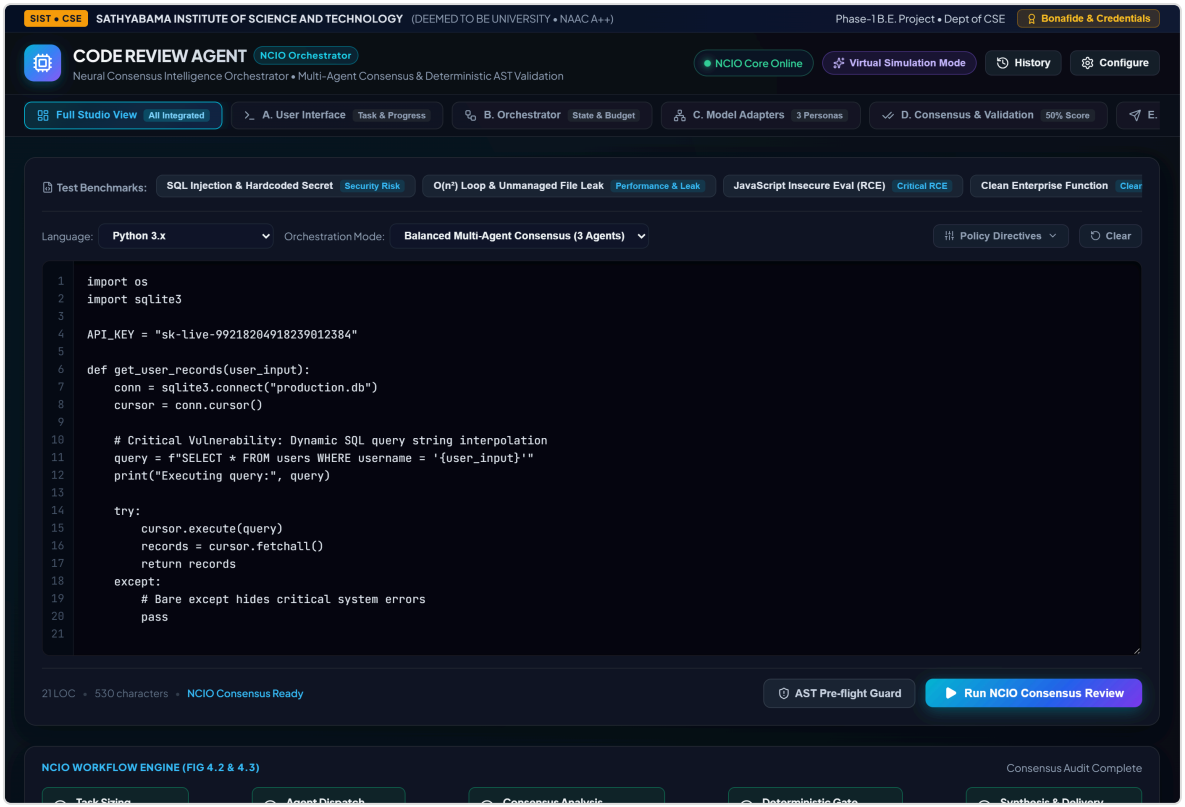


Fig 4.9: Full Integrated NCIO Studio Workspace (End-to-End System)

4.3 DESCRIPTION OF SOFTWARE IMPLEMENTATION AND TESTING PLAN

4.3.1 Implementation Plan

1. Create a repository and define configuration for providers, timeouts, and call budgets.
2. Implement request validation and a task-state model.
3. Build one model adapter and establish a baseline response flow.
4. Add additional adapters behind a common interface.
5. Implement response normalization and structured comparison.
6. Add validation hooks for task-specific checks and safe tool execution.
7. Create a user interface and status reporting.
8. Add logging, metrics, error handling, and configuration documentation.
9. Run automated tests and resolve defects before any deployment.

4.3.2 Testing Levels

Table 4.2: Testing Levels

Test level	Purpose	Example
Unit testing	Verify individual functions and modules.	Input validation; response normalization.
Integration testing	Check communication between modules.	Orchestrator calls adapter and receives normalized output.
System testing	Check complete user workflow.	Submit task and receive final response.
Failure testing	Verify recovery from faults.	Timeout, rate limit, malformed output, provider unavailable.
Security testing	Check trust boundaries and permissions.	Prompt injection, secret leakage, unauthorized tool call.
Usability testing	Check whether users understand the interface.	Progress status, error message, uncertainty label.

4.3.3 Consensus and Validation Workflow

The proposed workflow begins by generating candidate responses only when the task warrants the additional expense. Candidates are normalized into fields such as answer, assumptions, evidence references, and confidence explanation. The system then checks whether they agree on the core result. If they disagree, it identifies the specific claims in conflict and applies an available validator.

For code tasks, validation includes syntax checks, unit tests, static analysis, or execution in a sandbox with restricted permissions. If validation resolves the conflict, the final response summarizes the supported result. If the conflict remains, the system states the uncertainty and routes the result to a human reviewer.

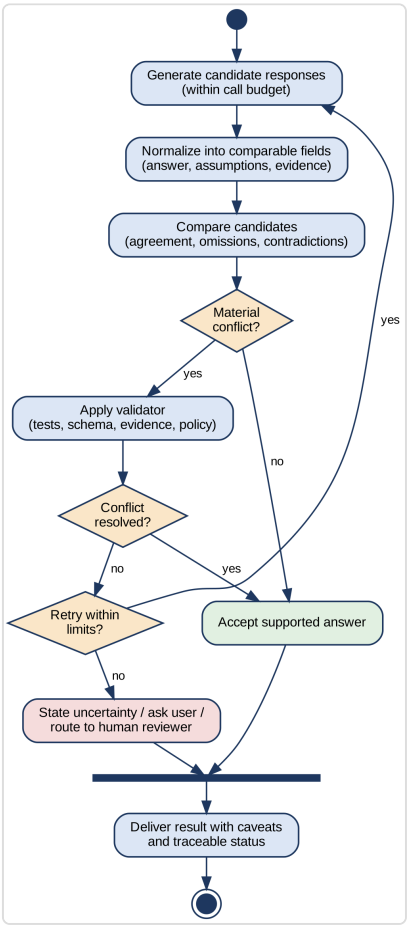


Fig 4.3: Consensus and Validation Workflow

4.4 PROJECT MANAGEMENT PLAN

Project management combines scope control, milestone tracking, risk review, and quality assurance. The work should be organized into deliverables that can be demonstrated and evaluated. The project lead maintains the issue list, documents design decisions, and checks that each milestone satisfies its acceptance criteria.

Table 4.3: Project Phases and Acceptance Criteria

Phase	Indicative duration	Acceptance criteria
Requirements and literature review	Week 1	Scope, risks, requirements, and references reviewed.
Architecture and interface design	Week 2	Module responsibilities and data flow documented.
Baseline implementation	Weeks 3–4	One end-to-end request completes successfully.
Multi-model orchestration	Weeks 5–6	Provider adapters and normalized outputs work.
Validation and safety	Weeks 7–8	Budget, stop rules, validation, and permission checks added.
Testing and refinement	Weeks 9–10	Test suite executed; high-severity defects addressed.
Documentation and demonstration	Weeks 11–12	Report, user guide, limitations, and demo prepared.

4.4.1 Quality Assurance

- Use version control and meaningful commit messages.
- Review configuration and dependency changes.
- Maintain repeatable test cases for core workflows.
- Track defects by severity and verify fixes.
- Document known limitations and avoid unsupported performance claims.

4.5 FINANCIAL REPORT ON ESTIMATED COSTING

The estimate below is an illustrative planning budget for a small academic prototype, not a quotation or a record of actual expenditure. Costs vary with provider pricing, usage, hosting choice, and whether free student or open-source resources are available. Currency is Indian rupees (INR).

Table 4.4: Estimated Implementation Cost

Item	Estimated cost (INR)	Notes
Development tools	0	Open-source editor, Git, and testing tools.
Local development machine	0 incremental	Assumes an existing personal or institutional computer.
Model API experimentation	1,000–5,000	Usage-dependent; enforce request and token budgets.
Prototype hosting	0–2,000	Local execution or a low-cost temporary deployment.
Database and logging	0–1,000	Local database or free/low-tier managed service.
Contingency	500–1,500	Unexpected usage or deployment needs.
Illustrative total	**1,500–9,500**	Planning range only; excludes labour and taxes.

4.5.1 Cost-control Measures

Use a low-cost model for classification and simple tasks, reserve stronger models for tasks that justify them, cache repeatable non-sensitive results where suitable, cap maximum tokens, and stop when the expected value of another model call is low. Usage dashboards and alerts should be configured before exposing the service to multiple users.

4.6 TRANSITION / SOFTWARE TO OPERATIONS PLAN

Transition begins only after the prototype passes the agreed tests. The deployment package should include the application, dependency lock files, environment configuration examples, and a rollback procedure. Secrets must be supplied securely at deployment time. The system should first be released to a limited test group, with logs and cost monitoring enabled.

Operational readiness checks include authentication, rate limits, access permissions, backup or recovery requirements, provider status handling, privacy notice, data-retention policy, and a process for reporting defects. A release should be rolled back if critical failures, unexpected tool actions, or unacceptable cost increases occur. Changes to prompts, models, and validation logic should be versioned and tested.

CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENTS

5.1 CONCLUSION

This report examined Agentic AI as an approach for building systems that plan, use tools, observe results, and revise actions toward a defined goal. The Neural Consensus Intelligence Orchestrator was presented as a proposed case study that combines modular model access with response comparison and validation. The requirements and architecture emphasize bounded execution, explicit error handling, traceability, and privacy-aware logging.

A central conclusion is that consensus should not be confused with truth. Agreement among models can be useful, but shared errors remain possible. Reliable agentic applications need evidence checks, task-specific tests, clear permissions, stopping rules, and a way to communicate unresolved uncertainty. The report provides a design and evaluation plan; empirical performance claims should only be made after the proposed prototype is implemented and tested.

5.2 FUTURE ENHANCEMENTS

Future extensions include integration with continuous integration/continuous deployment (CI/CD) pipelines (e.g. GitHub Actions), multi-file cross-repository dependency tracing, dynamic sandboxed unit test execution, and reinforcement learning from human feedback (RLHF) to refine agent role specializations over time.

REFERENCES

1. Brown, T. B., Mann, B., Ryder, N., et al. “Language Models are Few-Shot Learners.” *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
2. Vaswani, A., Shazeer, N., Parmar, N., et al. “Attention Is All You Need.” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
3. Wei, J., Wang, X., Schuurmans, D., et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
4. Yao, S., Zhao, J., Yu, D., et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” *International Conference on Learning Representations*, 2023.
5. Lewis, P., Perez, E., Piktus, A., et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
6. Ouyang, L., Wu, J., Jiang, X., et al. “Training Language Models to Follow Instructions with Human Feedback.” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
7. Sutton, R. S., and Barto, A. G. *Reinforcement Learning: An Introduction*. 2nd ed., MIT Press, 2018.
8. Wooldridge, M. *An Introduction to MultiAgent Systems*. 2nd ed., John Wiley & Sons, 2009.

9. Bellifemine, F. L., Caire, G., and Greenwood, D. *Developing Multi-Agent Systems with JADE*. John Wiley & Sons, 2007.
10. Russell, S., and Norvig, P. *Artificial Intelligence: A Modern Approach*. 4th ed., Pearson, 2021.
11. Mitchell, T. M. *Machine Learning*. McGraw-Hill, 1997.
12. Bergstra, J., and Bengio, Y. "Random Search for Hyper-Parameter Optimization." *Journal of Machine Learning Research*, vol. 13, 2012, pp. 281–305.
13. NIST. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology, 2023.
14. Amershi, S., Begel, A., Bird, C., et al. "Software Engineering for Machine Learning: A Case Study." *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*, 2019, pp. 291–300.
15. Sculley, D., Holt, G., Golovin, D., et al. "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems*, vol. 28, 2015.
16. Dean, J., and Ghemawat, S. "MapReduce: Simplified Data Processing on Large Clusters." *Communications of the ACM*, vol. 51, no. 1, 2008, pp. 107–113.